

GUIA DE INSTALAÇÃO DO GIT E ENVIO DE ATIVIDADES PARA O GITHUB

Turma 2 do Módulo de Lógica de Programação e Algoritmos

Este guia apresenta o procedimento completo para instalação do Git, criação de conta no GitHub, cópia da estrutura de pastas do repositório da turma e envio das atividades para um repositório próprio de cada aluno.

Observação importante: o repositório indicado a seguir deve ser utilizado apenas como modelo de estrutura de pastas. Os arquivos das atividades não devem ser enviados para ele. Após a cópia da estrutura, cada aluno deve criar um repositório próprio no GitHub e enviar os arquivos para esse novo repositório.

Repositório modelo da turma: <https://github.com/helidadbatista/Iteam-Logica-de-programacao-turma2>

1. O que é o Git e o que é o GitHub

O Git e o GitHub são ferramentas distintas que funcionam em conjunto. O Git é um programa instalado no computador, responsável por registrar o histórico de alterações dos arquivos de um projeto, e funciona mesmo sem conexão à internet. O GitHub é uma plataforma na internet que armazena uma cópia do repositório Git, permitindo o acesso a partir de qualquer computador e o compartilhamento do projeto com outras pessoas.

Em resumo, o Git organiza e registra o trabalho no computador do aluno, enquanto o GitHub disponibiliza esse trabalho na internet.

2. Instalação do Git (Windows)

1. Acessar o endereço git-scm.com/download/win. O download do instalador é iniciado automaticamente.
2. Abrir o arquivo executável baixado.
3. Selecionar a opção Next nas telas seguintes, mantendo as opções padrão.
4. Ao final da instalação, abrir o Git Bash ou o Prompt de Comando.
5. Confirmar a instalação com o comando:

```
git --version
```

Motivo: a instalação do Git é necessária antes de qualquer outro procedimento, pois todos os comandos seguintes dependem do programa estar presente no computador.

3. Configuração de identidade no Git

A cada alteração salva, chamada de commit, o Git registra o nome e o e-mail configurados no computador. Por esse motivo, é necessário configurar essas informações uma vez em cada máquina utilizada.

Primeiro, verificar os dados já configurados, especialmente em computadores de laboratório, que podem conter informações de outro usuário:

```
git config --global user.name
git config --global user.email
```

Caso os dados exibidos não correspondam ao aluno, ou caso nenhuma informação seja exibida, configurar com os dados corretos:

```
git config --global user.name "Nome Completo do Aluno"
git config --global user.email "email@exemplo.com"
```

Motivo: o nome e o e-mail configurados aparecem em todos os commits realizados. O uso do mesmo e-mail cadastrado no GitHub faz com que os commits sejam vinculados ao perfil do aluno.

Observação: o parâmetro `--global` aplica a configuração a todos os repositórios utilizados naquele computador. Em laboratórios compartilhados entre turmas, recomenda-se configurar sem esse parâmetro, diretamente dentro da pasta do projeto.

4. Criação de conta no GitHub

1. Acessar o endereço github.com/signup.
2. Preencher e-mail, senha e nome de usuário.
3. Confirmar o e-mail cadastrado.

Motivo: a conta no GitHub permite armazenar os repositórios na internet e é o link que será enviado à professora para correção das atividades.

5. Configuração de autenticação

O GitHub não aceita mais a senha da conta para envio de arquivos pela linha de comando, sendo necessário um token de acesso pessoal.

1. No GitHub, acessar a foto do perfil, no canto superior direito, e selecionar Settings.
2. No menu lateral, acessar Developer settings, em seguida Personal access tokens e Tokens (classic).
3. Selecionar Generate new token, marcar a permissão "repo" e gerar o token.
4. Copiar e guardar o token gerado, pois ele é exibido apenas uma vez.

Motivo: o token substitui a senha da conta nas operações realizadas pela linha de comando e será solicitado no primeiro envio de arquivos.

6. Cópia do repositório da turma

O repositório da turma apresenta a estrutura de pastas que deve ser seguida nas atividades, com uma pasta por aula e uma subpasta por exercício. Essa estrutura deve ser copiada para o computador do aluno.

1. Abrir o terminal e acessar a pasta onde o projeto será salvo, por exemplo:

```
cd Documentos
```

2. Copiar o repositório da turma:

```
git clone https://github.com/helidadbatista/Iteam-Logica-de-programacao-turma2.git
```

3. Acessar a pasta criada:

```
cd Iteam-Logica-de-programacao-turma2
```

Motivo: o comando `git clone` baixa uma cópia completa do repositório, incluindo arquivos, pastas e histórico, para o computador do aluno.

7. Remoção do histórico do repositório copiado

A pasta copiada permanece conectada ao repositório da professora e carrega o histórico de commits dela. Antes de criar o repositório próprio, essa conexão deve ser desfeita.

O comando para apagar a pasta `.git` muda conforme o terminal utilizado no laboratório. No Prompt de **Comando (cmd)**, usado nesta turma, o comando é:

```
rmdir /s /q .git  
git init
```

Caso o terminal utilizado seja o **Git Bash**, o comando equivalente é:

```
rm -rf .git  
git init
```

Caso o terminal utilizado seja o **PowerShell**, o comando equivalente é:

```
Remove-Item -Recurse -Force .git  
git init
```

Motivo: o comando de remoção apaga a conexão com o repositório da professora e o histórico de commits copiado junto com os arquivos. O comando `git init` cria um repositório novo e vazio na mesma pasta, preservando os arquivos e a estrutura, porém com o histórico zerado.

Observação: esse procedimento remove apenas a pasta oculta `.git`, responsável pelo controle de versão. Os arquivos e pastas visíveis permanecem inalterados.

Observação: caso o comando `git init` já tenha sido executado por engano, antes da remoção da pasta `.git`, o procedimento pode ser corrigido normalmente: basta apagar a pasta `.git` agora e repetir o `git init` em seguida. A ordem não altera o resultado. A única atenção necessária é que, se o aluno já tiver executado `git add` e `git commit` nesse meio tempo, esses commits serão apagados junto com a pasta `.git`, sendo necessário refazer o `git add` e o `git commit` depois do novo `git init`.

8. Criação do repositório próprio no GitHub

1. No GitHub, selecionar o símbolo de adição, no canto superior direito, e em seguida New repository.
2. Definir um nome para o repositório.
3. Selecionar a visibilidade Public ou Private, conforme orientação da professora.
4. Não selecionar a opção "Add a README file".
5. Selecionar Create repository.

Motivo: a criação do repositório sem arquivo inicial evita conflito no envio dos arquivos já existentes no computador do aluno.

9. Conexão da pasta local ao repositório criado

Na página do repositório criado, copiar o endereço HTTPS, disponível no botão Code, e executar o seguinte comando dentro da pasta do projeto:

```
git remote add origin ENDERECO_DO_REPOSITORIO.git
```

Motivo: esse comando informa ao Git para qual repositório do GitHub os arquivos devem ser enviados.

Em seguida, conferir se o endereço foi registrado corretamente:

```
git remote -v
```

O comando deve exibir o endereço do repositório do aluno, associado a fetch e a push. Caso o endereço exibido ainda seja o do repositório da professora, a etapa 7 não foi concluída corretamente.

Observação: se aparecer a mensagem "error: remote origin already exists", significa que já existe um remoto origin configurado, geralmente porque a pasta .git não foi apagada na etapa 7. Nesse caso, em vez de git remote add, deve ser usado:

```
git remote set-url origin ENDERECO_DO_REPOSITORIO.git
```

Como alternativa, também é possível remover o remoto existente e adicionar o novo em seguida:

```
git remote remove origin  
git remote add origin ENDERECO_DO_REPOSITORIO.git
```

Depois de qualquer uma das correções, repetir o comando git remote -v para confirmar que o endereço exibido é o correto antes de seguir para o envio dos arquivos.

10. Envio dos arquivos

A cada atividade concluída, a seguinte sequência de comandos deve ser executada:

```
git add .  
git commit -m "Aula 02, Exercício 01"  
git push -u origin main
```

git add . seleciona todos os arquivos novos ou alterados para o próximo commit.

git commit -m "mensagem" registra as alterações selecionadas, acompanhadas de uma mensagem descritiva.

git push -u origin main envia os commits salvos para o repositório no GitHub. O parâmetro -u origin main é necessário apenas no primeiro envio.

Observação: no primeiro envio, o Git solicita usuário e senha. Deve ser utilizado o nome de usuário do GitHub e, no campo de senha, o token pessoal gerado na etapa 5.

Observação: caso a branch local seja identificada como master, e não main, deve ser utilizado o comando `git push -u origin master`. Para verificar o nome da branch local, executar o comando `git branch`.

11. Confirmação e envio do link

1. Atualizar a página do repositório no navegador e confirmar a presença dos arquivos e pastas.
2. Copiar o endereço da página do repositório, sem a extensão `.git`.
3. Enviar o endereço copiado à professora.

12. Organização das pastas

A estrutura de pastas deve seguir o padrão do repositório modelo, com uma pasta por aula e uma subpasta por exercício:

```
AulaXX/  
  Ex01/  
    algoritmo.alg  
    algoritmo.py  
    fluxograma.png  
  Ex02/  
  Ex03/
```

- Evitar espaços e acentos nos nomes de arquivos e pastas.
- Manter no repositório os exercícios das aulas anteriores.
- Realizar commits com frequência, um a cada exercício concluído.

13. Problemas frequentes

Solicitação repetida de usuário e senha: indica que o token de acesso pessoal não foi gerado, ou que a senha da conta foi utilizada em seu lugar.

Mensagem "Permission denied" no envio dos arquivos: geralmente indica que o repositório remoto configurado ainda é o da professora. Verificar com o comando `git remote -v` e, se necessário, repetir a etapa 9.

Mensagem "nothing to commit": indica que o comando git add . não foi executado antes do commit. O comando git status exibe os arquivos pendentes.

Mensagem "src refspec main does not match any": indica que a branch local possui outro nome, geralmente master. Verificar com o comando git branch.

Mensagem "remote origin already exists": indica que a etapa 7 não foi concluída antes da etapa 9. Usar git remote set-url origin, conforme explicado na etapa 9, em vez de apagar e refazer o clone.

Instituto Tecnológico Educacional da Amazônia (ITEAM). Professora Héliida Dias Batista Xerfan.